Mountain Tourism Complex Platform

MVP scope · Version 1 · October 2026

The platform is for mountain tourism complexes: one site that runs everything a mountain guest needs. That means a hotel, chalets and huts, restaurants from the valley to the summit, a gear shop and rental, sauna, spa and sports, lifts, and transport. Today most complexes run each part on a separate system or on paper. Nobody sees the guest's full bill in one place, and weather decisions are made by phone calls.

The long-term product is a web SaaS for many complexes. Everything runs in the browser on laptops, tablets and phones, with no apps to install. This MVP is the first step: about 12 weeks of work, with a foundation the full product can grow on without a rewrite.

This document is the contract scope for the MVP. Anything not listed here is out of scope unless we both agree to it in writing.

1. What the MVP has to prove

> **Goal:** a pilot complex can run one full winter or summer day on it. Guests arrive by train or plane and get picked up, check in, rent gear, ride the lifts, eat on the mountain, use the sauna, and check out with one correct bill. When the wind picks up at the summit, staff see it before the guests do.

I will pilot the MVP with 2 or 3 real complexes and build the full product from what they tell me. Anything that does not serve that one day is left for later.

2. What a complex looks like

Use this layout for the demo seed data. The altitudes are examples, but the idea is real: every building sits at its own altitude, and the weather at each one is different.

Building	Altitude	What happens there
Valley station	1,050 m	Parking, bus and train stop, ticket desk, rental shop, lower end of the cable car
Main hotel	1,200 m	Rooms, reception, main restaurant, sauna, thermal bath, spa, game room, gym
Chalets	1,250 m	Family chalets, booked like rooms
Mid station	1,800 m	Self-service restaurant, ski school meeting point, chairlift start
Mountain hut	2,100 m	Dormitory beds sold one by one, simple meals, start of guided hikes
Summit station	2,600 m	Panorama restaurant, viewing platform, top of the cable car

3. Users and roles

Role	Works in	Main jobs
Super admin (me)	Super admin page	Create complexes and their owner accounts, activate or deactivate

Role	Works in	Main jobs
Owner	Everything	Settings, buildings, outlets, staff, prices, weather limits, reports
Manager	Assigned outlets	Products, prices, voids, outlet reports
Front desk	Hotel, chalets, hut	Reservations, check-in and check-out, folios, payments
Cashier	Restaurants, hut, shop	Orders, payments, charge to room
Rental desk	Rental shop	Rent out, returns, damage fees
Service staff	Spa, sauna, sports, ski school	Slots, bookings, own schedule, headcount on guided outings
Transport desk	Transport	Shuttles, taxi trips, pickups, vehicles and drivers
Lift operator	Lift stations	Lift status from a phone

Drivers do not log in for the MVP. They get a printed passenger list.

4. Core concepts

These names should show up in the database and the code as they are here, so we talk about the same things.

Tenant — One complex. Every table carries `tenant_id`. Each staff account belongs to one tenant, so the tenant is known from the login (no subdomains in the MVP).

Building — A place in the complex with a type, map position and altitude. Every building is also a weather point.

Outlet — A business unit in a building: restaurant, hut kitchen, shop, rental, spa, sports, transport, lifts. It has its own products, staff and reports.

Season — Winter, summer or closed, with dates. Prices, menus and services can differ per season.

Stay, Board plan — A stay is one visit in a room, chalet or hut bed. The board plan (room only, B&B, half board, full board) says which meals are included.

Folio, Charge — The folio is the bill of a stay. A charge is one line on it and always records its outlet, tax and source (order, booking, rental, room night).

Payment — Cash or card, recorded by staff. One folio can take several payments.

Service, Slot — A bookable hotel service (sauna, spa treatment, ski lesson, guided hike) and one time slot of it with a capacity and a staff member.

Rental — One rented item (skis, boots, bike) from out to return.

Transport type — How people move: own transport, lifts, or external. See module G.

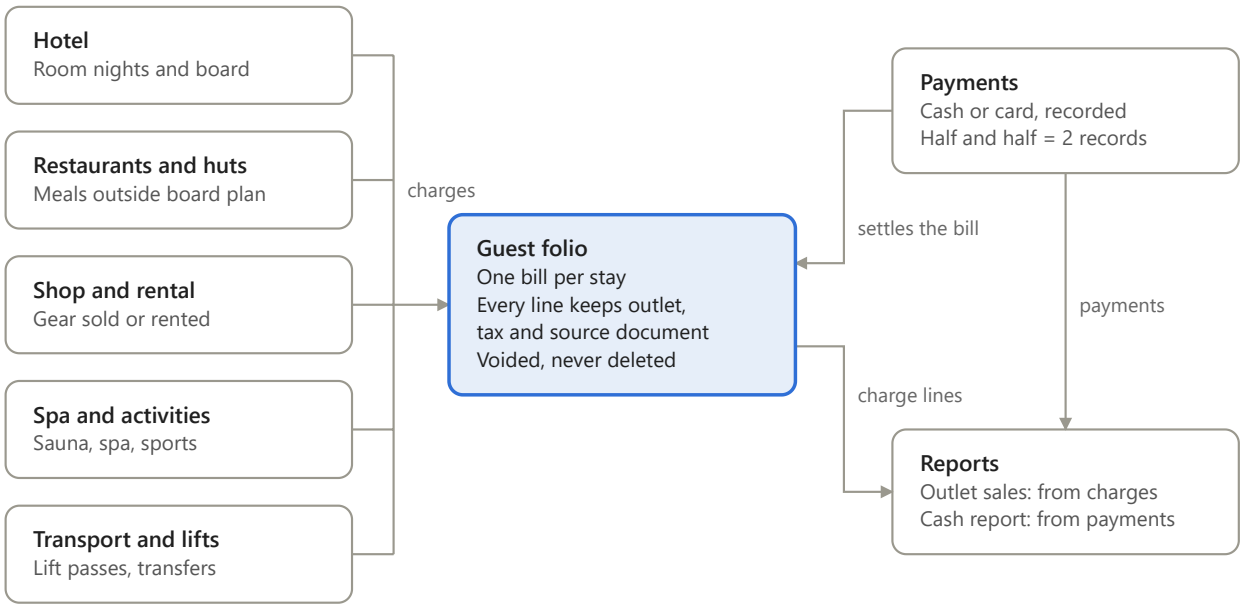
Trip — One run of own transport: a scheduled shuttle departure, a taxi ride or a pickup, with a vehicle, a driver and seats.

Lift, Lift pass — A lift between two buildings with a status, and a pass product that lets a guest ride.

Weather limit, Warning — A limit set by the complex (for example wind gusts at a lift station), and the warning raised on everything it affects.

> **One hard rule:** money is append-only. A charge or payment is never edited or deleted. It is voided, with a reason, a user and a timestamp. A weather cancellation is a void with the reason “weather”.

Every outlet bills one guest folio, and reports still split by outlet



Outlet revenue is read from charge lines and cash from payments. A bill split between card and cash, or a lesson cancelled for weather, never blurs what each outlet earned.

5. Modules

Each module ends with a “done when” check. That is what I test on my VPS before releasing the milestone payment.

A. Core and super admin

- Login with email and password. Owners and managers reset staff passwords themselves; no email sending in the MVP
- Super admin page: create a complex and its owner account, activate or deactivate it
- Buildings with type, altitude and map position; outlets inside buildings
- Staff accounts with role and outlet access
- Seasons with dates; settings for currency, tax rates, timezone and units (°C and km/h, or °F and mph)
- Audit log for every charge, payment and void

> **Done when:** two complexes on the same server cannot see each other's data, proven by an automated test, not by clicking around.

B. Hotel, chalets and huts

- Room types and rooms in any building. Hut dormitories sell single beds, not whole rooms
- Price per night per season
- Board plans with a price per person per night: room only, B&B, half board, full board
- Reservations: dates, adults and children, room or bed, board plan, deposit, and how the guest arrives (car, train, plane, bus, with time and station)

- Calendar grid by building, room and date
- Check-in opens the stay and its folio; room nights and board are posted automatically every night

> **Done when:** a 3-night half-board stay posts 3 room nights and 3 board charges, the included dinners are not charged again, and checkout ends at a zero balance.

C. Guest folio and checkout

- Folio shows every charge with outlet, date, amount and tax
- Any outlet can charge to a room by room number or guest name
- Payments: cash or card (recorded only, no gateway), several payments per folio
- Void a charge with a reason, manager role only
- Checkout is blocked until the balance is zero; printable invoice (browser print or PDF)

> **Done when:** a summit lunch and a ski lesson are charged to the room, the lesson is cancelled for wind, the guest pays the rest half card and half cash, the folio closes at zero, and each outlet's report shows only its own sales.

D. Restaurants, huts and shop

- Outlets at any altitude: valley restaurant, mid-station self-service, summit restaurant, hut kitchen, bar, gear shop
- Products with categories, price and tax; menus shown or hidden by season
- Tablet order screen, a note per item, table number
- Board check: when the cashier picks a hotel guest, the screen shows whether this meal is included in their board plan
- Packed lunches for hikers and skiers, ordered the evening before, with a printed list for the kitchen in the morning
- Printable kitchen ticket and receipt (browser print, 80 mm)
- Pay now (cash or card) or charge to room; cashier shift open and close with cash count

> **Done when:** a half-board guest's dinner is recorded as included, and a bottle of wine at the same table is charged to the room.

E. Equipment rental

- Item types set by the complex: skis, snowboards, boots, helmets, poles, sleds, bikes, e-bikes, hiking poles and so on
- Each item with size and an item code; price per day by season; deposit
- Rent out to a hotel guest or a walk-in customer, return, mark damaged with a fee
- List of everything currently out, and overdue items highlighted

> **Done when:** one item cannot be rented twice at the same time, and a late return adds the extra day to the bill.

F. Hotel services and activities

One booking engine for every service. The complex adds services itself; we do not build a separate module per service.

- Service types: sauna, thermal bath, spa treatments, game room, gym, climbing wall, ski school, guided hikes and ski tours, or anything else the complex adds

- Each service: duration, capacity, price (or included in the stay), minimum age, the staff it needs (therapist, instructor, guide), indoor or outdoor, and the building it happens at
- Time slots by weekday and season; staff assigned to slots
- Bookings from the front desk or the service desk, pay now or charge to room
- Guided outings: participants are checked out at the start and checked back in at the end. An outing stays open until everyone is marked back

> **Done when:** a sauna slot cannot go over capacity, one instructor cannot be in two lessons at once, a cancelled booking removes its charge, and an outing with a missing participant cannot be closed.

G. Transport and lifts

The complex keeps a list of transport types. Each type belongs to one of three groups, and the system treats each group differently.

Group	Types	What the system does
Own transport	Shuttle bus, taxi, private car, bicycle, motorbike	Vehicles, drivers, schedules, trips and bookings. Bicycles and motorbikes go out through rental (module E)
Lifts	Cable car, gondola, chairlift, funicular, underground funicular, elevator	Status, opening hours, lift passes
External	Plane, train, metro and underground, intercity bus	Records the guest's arrival and departure, and creates a pickup or drop-off trip

- Own transport: vehicles with seats, drivers, scheduled shuttle departures and on-demand taxi trips, routes with stops on a map, bookings, printable passenger list
- Routes are drawn on the map by staff: the stops and the path between them. No routing service, because lift lines and trails are not in road routers anyway
- No driver or vehicle can be on two trips at the same time, and a trip cannot be booked over its seats
- Pickups: a reservation that says “arrives by train at 14:20 at the valley station” creates a pickup trip for the transport desk to plan. The same goes for departures
- Lifts: each lift runs between two buildings, with type, opening hours and a status (open, closed, on hold) that the lift operator changes from a phone. Every change is kept with time and reason
- Lift passes: products such as 1 day, 6 days, pedestrian or summer, adult or child. Sold at any POS or the front desk and charged to room. No gate hardware in the MVP

> **Done when:** a driver cannot be double-booked, a train arrival on a reservation creates the pickup trip, and a lift set to closed shows on the lobby screen within a minute.

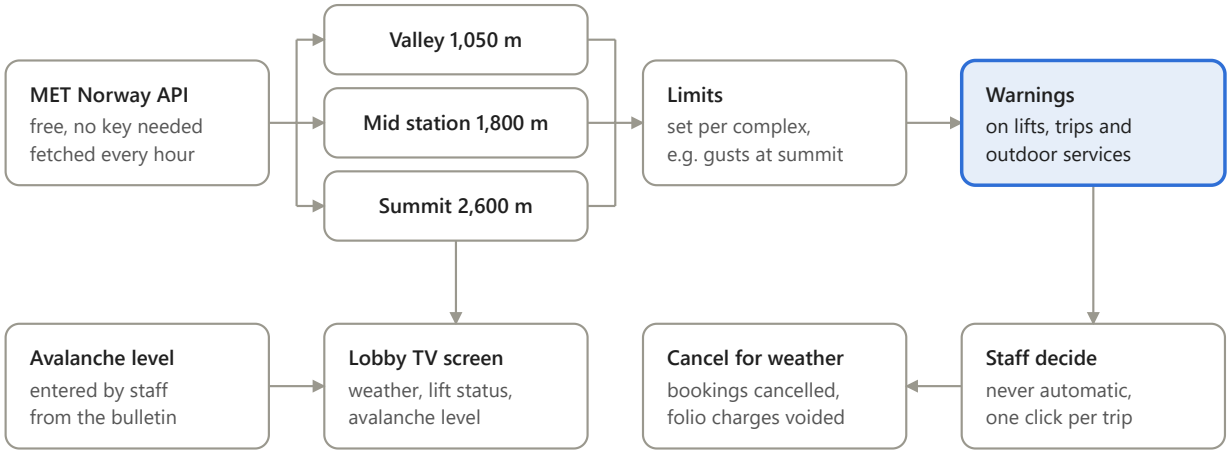
H. Weather

Weather is a core feature, not a widget. In the mountains it decides what runs today.

- Every building and lift station is a weather point with coordinates and altitude. The valley and the summit get their own forecasts
- Data from the MET Norway Locationforecast API (`api.met.no`). It is free, allows commercial use, needs no key, and takes altitude into account. A background job fetches it every hour and stores it, following MET Norway's rules on caching and the User-Agent header. If the API is down, the last data stays on screen with its time

- Shown per point: now, the next 48 hours by hour, and about 9 days ahead. That covers temperature, wind and gusts, wind direction, precipitation shown as snow or rain from the temperature at that altitude, cloud and fog, humidity and UV
- Limits set by each complex per weather point, for example wind gusts at the summit station, precipitation per day, fog, or very low temperature
- Weather screens show “Weather data from MET Norway” and maps show the OpenStreetMap credit, as their free licences require
- When the forecast crosses a limit, a warning is attached to everything affected in that time window: lifts, trips, and outdoor service slots at that building. Warnings show on the dashboard and on the transport and service screens
- People decide, the system never cancels on its own. “Cancel for weather” on a trip, a slot or a lift day cancels its bookings, voids their charges with the reason “weather”, and prints the list of guests to inform
- Avalanche danger level (1 to 5), entered by staff each morning from the official regional bulletin
- Lobby screen: a read-only page for a TV in the lobby or at the valley station, opened by a private link with no login. It shows weather at valley and summit, lift status, the avalanche level, and today's activities

Weather warns staff early; people decide, the system fixes the bills



Each building is a weather point. Altitudes are the example ones from section 2.

> **Done when:** lowering a limit makes warnings appear on the matching lift, trips and outdoor slots, and a weather cancellation leaves every affected folio correct.

I. Owner dashboard and reports

- Today: occupancy by building, arrivals and departures with their transport, revenue by outlet, rentals out, open folios, lift status, weather warnings
- Reports for a date range: revenue by outlet, by service and by payment method, weather cancellations, CSV export

> **Done when:** report totals match the folio charges plus direct sales for the same dates, to the cent.

6. Hosting and CI/CD

There is no domain at this stage, and no Docker. The MVP is installed directly on my own VPS and opens at the server's IP address.

Server

- My VPS: Ubuntu 24.04 LTS, 2 CPU cores, 4 GB RAM, 100+ GB free disk, outgoing internet for the weather API and map tiles. Node.js 22, PostgreSQL and Nginx are already installed.
- **Other live websites already run on this server and must not be touched.** The platform gets its own Linux user, its own PostgreSQL database and user, its own internal port and its own Nginx server block. Builds happen on GitHub, not on the server, to keep memory free.
- You get your own SSH user with the rights the setup needs, not the root password.
- The app runs directly on the server. PM2 or systemd keeps it running and restarts it after a crash or a reboot. If you choose Laravel, PHP-FPM instead of Node.js.
- Background jobs (weather fetch, nightly posting) run through the app's scheduler or cron.
- A setup script or step-by-step guide that prepares a fresh VPS from zero, so I can rebuild the server without you.
- All settings in an `.env` file on the server, never in GitHub. Nothing hard-coded to the IP address.
- Address for now: a free sslip.io hostname built from the server IP, with a free Let's Encrypt certificate, or a separate port. No domain to buy.
- Adding a real domain later must be a change to Nginx and `.env` only, not to the code.
- Nightly database backup with `pg_dump` to a folder on the server, last 7 kept, and one restore tested before handover.

CI/CD with GitHub Actions

- On every pull request: install, lint, run the tests and build. A failed check blocks the merge.
- On merge to `main`: build on GitHub, copy the release to the VPS over SSH into a new release folder, run the database migrations, switch to the new release and reload the app without downtime.
- Health check after every deploy. If the new version does not answer, the pipeline switches back to the previous release by itself.
- The last 5 releases stay on the server for a quick manual rollback.
- The SSH key lives in GitHub Secrets. Nothing secret goes into the repository.
- Runs on GitHub's free Actions minutes. No paid CI service.
- Optional: a test copy on the same VPS on another port (for example `:8080`) with its own database, updated automatically from a `develop` branch.

7. Technical rules

These are not negotiable. The rest of the stack is open for discussion. My preference is Next.js, NestJS (or Laravel) and PostgreSQL.

- **Free services only.** Weather from MET Norway, maps from free OpenStreetMap-based tiles with Leaflet or MapLibre. No paid APIs, no API keys, no trial plans that start charging later. Anything else needs my OK first.
- **Tenant isolation in one place.** Postgres row-level security or a scoped repository that every query goes through, not a `where tenant_id = ?` repeated in controllers.
- **Money in integer minor units** (cents), never floats.
- **Permissions checked on the API**, not only hidden in the UI.

- **Weather called from the server only,** cached and shared by all screens, never fetched by each browser.
- **Web only.** Everything runs in the browser. No native apps.
- **API documented** with OpenAPI (Swagger). The guest web portal later will use the same API.
- **Timezone, currency and units per tenant.** Nightly posting, reports and “today” use the tenant's local time.
- **Devices.** Staff screens work on a 10-inch tablet and a laptop; the lift operator page works on a phone.

8. Milestones and payments

Each payment is released when the milestone is running on my VPS and passes the “done when” checks in section 5.

#	Milestone	What gets delivered	Weeks	Payment
1	Foundation	Data model and short architecture note for my review, server setup on my VPS, CI/CD with automatic deploy, login, tenants, buildings with altitude, outlets, staff and roles, seasons, super admin page	1–2	15%
2	Hotel and folio	Rooms, chalets and hut beds, season prices, board plans, reservations with arrival details, calendar, check-in and check-out, nightly posting, folio, payments, voids, invoice	3–5	25%
3	POS, rental, services	Restaurant, hut and shop POS with board check, packed lunches, equipment rental, the service booking engine with slots and staff, guided outing headcount	6–8	25%
4	Transport, lifts, weather	Transport types, vehicles, drivers, routes on the map, shuttles, taxi trips, pickups, lifts and status, lift passes, weather points, forecasts, limits, warnings, cancel for weather	9–10	20%
5	Dashboard and handover	Owner dashboard and reports, lobby screen, avalanche level, backups, bug fixing, handover (section 11)	11–12	15%

After the final handover, bugs in anything covered by this scope are fixed free for 30 days.

9. After the MVP

None of this is in the contract, but the MVP design must not block it. If the MVP goes well, I want to continue with the same developer.

- Own domain, a subdomain per complex, proper SSL
- Self-signup and monthly subscriptions, online payments, a public booking website
- Guest web portal that works on any phone, no app to install: weather alerts, book services, lift passes, see the bill
- Lift gate and RFID card integration
- Offline mode for huts with weak signal
- SMS or WhatsApp weather alerts to guests
- Richer weather data such as snow depth and freezing level, from a paid provider if the pilots ask for it
- Booking real train and flight tickets through partners
- Channel manager and OTA sync, accounting export, multi-currency

10. Who provides what

From me: the VPS with SSH access, logo and colours, sample data from a real complex, and answers within 24 hours on working days. No API keys are needed: everything the MVP calls is free and open.

From you: all source code, migrations, seed data, the server setup script, the GitHub Actions workflows, and documentation. Code and IP for each milestone transfer to me when that milestone is paid.

There are no UI designs. A clean component library (shadcn/ui, Ant Design or similar) is fine. I care about fast, obvious screens for busy staff, often in gloves and in a hurry, more than about looks.

11. Handover checklist

- ☐ All code in my GitHub
- ☐ Running on my VPS, deployed by the CI/CD pipeline
- ☐ Server setup script or guide for a fresh VPS
- ☐ README with local setup and deploy steps
- ☐ `.env.example` with every variable explained
- ☐ API documentation (Swagger)
- ☐ Database restore and release rollback, each tested once
- ☐ Seed script that creates the demo complex from section 2
- ☐ One recorded walkthrough call of about an hour